Companion — Presentation Commander Server user guide

This module drives a Presentation Commander **Master Server** from a Stream Deck, or any other Bitfocus Companion surface, so you can route outputs, recall scenes, black out, send stage notes and step slides.

The README covers installing the module and pointing it at the server. This guide is about building a surface with it, and what to be careful of.

What this drives

This changes what an audience sees. A button press here routes live outputs on a running Master Server. There's no confirmation step and no undo.

Two things to know before you build a page:

- **The Master Server's automation API has no authentication.** That's why it binds `127.0.0.1` by default. If you've tunnelled or proxied it so Companion can reach it from another machine, anything else that can reach it can drive your show too.
- **Some of what this module reports can be up to 3 seconds out of date**, and some of it can be wrong while the connection's down (Feedbacks).

Connecting

Two config fields: **Master Server host** (default `127.0.0.1`) and **Automation API port** (default `9700`).

The default is loopback because that's where the server listens. **If Companion runs on a different machine**, you can't reach the server's API directly. The README has a section on that: it needs a tunnel or a proxy that adds authentication, not a config change.

Once it's connected the instance shows OK and the `connection_status` variable reads `Connected`.

The buttons you can build

Action	What it does
Route Output	send an output to a scene or a single source
Blackout Output	send an output to nothing
Recall Scene to Output	put a scene on one output
Send Note to Stage	push a message to the presenter
Next Slide / Previous Slide	step a Client Node's deck

All the dropdowns fill themselves from the live server, so you pick actual output, scene, source and client names rather than generated ids. They refresh by themselves as the server's lists change.

Worth knowing:

- **Route with "— Unrouted —" is the same as Blackout.** Both are there because they read differently on a button, but the command they send is identical.
- **Recall Scene affects one output**, not the whole rig. It isn't a global preset recall.
- **Send Note replaces the previous message.** There's one slot, not a queue. Send an empty message to clear it.
- **If the server had no outputs when the module last refreshed**, a new button's Output dropdown will start off blank. Open the dialog again once the server's up.

Slide buttons can succeed without moving anything

Next/Previous Slide go to the Master Server, which passes them on to the Client Node, **if that client's connected.** If it isn't, **the server steps its own internal counter and reports success anyway.**

This module can't tell the difference. The button behaves as though it worked while the projector sits there unmoved.

Put the "Client Node is online" feedback on those buttons. It's the only sign you get that the command has somewhere to go.

Feedbacks, and where they lie

There are two feedbacks, and both go green by default:

- **Output is routed to a specific source/scene:** the button lights when that output is on that target right now.
- **Client Node is online:** lights while that client is connected.

They keep showing the last known state when the server is unreachable

If the connection drops, the module marks itself failed and sets `connection_status` to `Disconnected`, **but the routing and client feedbacks carry on showing whatever they last saw.**

So a green "on air" button can stay green while the module has no idea what's actually on air.

Put `$(<instance>:connection_status)` somewhere you can see it on every page that has routing buttons. That's the variable you can trust.

Everything lags by up to 3 seconds

The module **polls** the server every 3 seconds; nothing is pushed live. If a route's changed from the server's own UI, or from another surface, the matching button won't light until the next poll.

Don't take a button that hasn't lit yet as a failed command.

Variables

Variable	Shows
`connection_status`	`Connected` / `Disconnected` — **the one to trust**
`client_count`	how many Client Nodes are online
`routed_<output>`	the scene or source name on that output, or `Unrouted`

`routed_*` variables only exist **after the first successful poll**. If the server's never been reachable they aren't defined at all, which is why a fresh install pointed at a stopped server shows nothing rather than `Unrouted`.

Building a surface that fails safe

1. **`connection_status` on every page** that has routing buttons (Feedbacks).
2. **"Client Node is online" on every slide button** (the buttons).
3. **`routed_<output>` as button text** on your route buttons. It reads back the name that's actually routed, so a stale feedback and a stale variable at least agree with each other.
4. **Keep Blackout next to Route** on the same page. It's the quickest way out of a wrong route.
5. Remember the 3-second poll before you press a button a second time (Feedbacks).

Troubleshooting

Symptom	Cause
Instance shows Connection Failure	Server not running, or not reachable at that host/port. Default is loopback-only (Connecting).
Companion is on another machine and can't connect	The API binds `127.0.0.1`. Needs a tunnel or proxy, not a config change (Connecting).
Dropdowns are empty	No successful poll yet. They fill from live server state (the buttons).
A new button's Output field is blank	The server had no outputs when the module last refreshed (the buttons).
Button lit green but nothing is on air	The connection may be down — feedbacks keep the last known state (Feedbacks). Check `connection_status`.
Slide button works but the deck doesn't move	The Client Node is offline and the server simulated it (the buttons).
A change I made on the server didn't light the button	Up to 3 seconds of poll lag (Feedbacks).
`routed_*` **variables don't exist**	No successful poll has ever happened (Variables).
Send Note replaced my previous message	One slot, not a queue (the buttons).
A Stream Deck button silently stopped working after a server update	The three repos share a protocol kept in sync by hand — this module is the one people forget. See DEVELOPING.md.

See also

- API.md: every action, feedback and variable, and the poll loop
- DEVELOPING.md: the three-repo protocol
- README: setup, Companion on another machine, and installing